SENTINELNET – AI-POWERED NETWORK INTRUSION DETECTION SYSTEM (NIDS)

Author: Upasana Prabhakar
Mentor: Dr. N Jagan Mohan

ABSTRACT: With the exponential growth of network infrastructure and the increasing sophistication of cyber threats, organizations face unprecedented challenges in protecting their digital assets. Traditional security measures struggle to keep pace with evolving attack vectors, including zero-day exploits, advanced persistent threats (APTs), and polymorphic malware. SentinelNet addresses these challenges by leveraging artificial intelligence and machine learning to build an intelligent Network Intrusion Detection System (NIDS) capable of real-time threat identification and automated response mechanisms. This research implements a comprehensive evaluation framework using two benchmark datasets: NSL-KDD and CICIDS2017, representing both legacy and modern attack scenarios. Through systematic experimentation with multiple machine learning algorithms—including Logistic Regression, Decision Trees, Random Forest, and Gradient Boosting—we analyze both binary classification (normal vs. attack) and multi-class classification (specific attack type identification) scenarios. Our preprocessing pipeline incorporates advanced techniques for handling class imbalance, feature engineering, and data normalization to optimize model performance. The experimental results demonstrate that ensemble methods, particularly Random Forest and Gradient Boosting, achieve superior detection rates with accuracies exceeding 99% on CICIDS2017 while maintaining computational efficiency. The system successfully identifies various attack types including DoS, Probe, U2R, R2L, DDoS variants, web attacks, and infiltration attempts. This work contributes to the cybersecurity domain by providing empirical evidence for the effectiveness of hybrid IDS approaches and establishing performance benchmarks for practical NIDS deployment in enterprise environments.

KEYWORDS: Network Intrusion Detection System (NIDS), Cybersecurity, Machine Learning, NSL-KDD Dataset, CICIDS2017 Dataset, Ensemble Methods, Network Security, Anomaly Detection, Real-Time Threat Detection

STATEMENT OF ORIGINALITY: This paper presents SentinelNet, a hybrid IDS combining signature-based and anomaly-based methods with advanced preprocessing, feature selection, and class-imbalance handling, applied to NSL-KDD and CICIDS2017 datasets.

1 INTRODUCTION

The proliferation of internet-connected devices and the rapid digitalization of critical infrastructure have fundamentally transformed the threat landscape in cybersecurity. Modern networks face a barrage of sophisticated attacks ranging from distributed denial-of-service (DDoS) operations to stealthy advanced persistent threats (APTs) that can remain undetected for months. Traditional perimeter defenses such as firewalls and antivirus software, while necessary, are no longer sufficient to protect against the dynamic and evolving nature of contemporary cyber threats. This paradigm shift necessitates intelligent, adaptive security solutions capable of learning from network behavior patterns and identifying anomalies indicative of malicious activity.

Network Intrusion Detection Systems (NIDS) serve as a critical component of defense-in-depth security strategies, providing continuous monitoring and analysis of network traffic to identify potential security breaches. Unlike preventive measures that block traffic based on predefined rules, NIDS offers visibility into network activities, enabling security teams to detect ongoing attacks, understand attacker methodologies, and respond with appropriate countermeasures. The integration of artificial intelligence and machine learning into NIDS architecture represents a significant advancement, enabling systems to automatically learn complex patterns from historical data and generalize to detect previously unseen attack variants.

SentinelNet emerges from this need for intelligent, autonomous security systems. The project aims to develop an AI-powered NIDS that combines the strengths of signature-based detection (fast identification of known threats) with anomaly-based detection (capability to identify novel attacks). By leveraging machine learning algorithms trained on comprehensive datasets representing both legacy and contemporary attack scenarios, SentinelNet seeks to achieve high detection accuracy while minimizing false positives that plague traditional rule-based systems.

The core objectives of this research include:

- **Automated Threat Detection:** Develop machine learning models capable of automatically distinguishing between benign network traffic and various categories of malicious activities without human intervention.
- **Real-Time Classification:** Implement efficient algorithms that can process and classify network flows in real-time, enabling immediate threat response and mitigation actions.

- **Feature Engineering and Optimization:** Extract and engineer relevant features from raw network packet data that maximize model discriminative power while reducing computational overhead.
- **Handling Class Imbalance:** Address the inherent class imbalance problem in intrusion detection where normal traffic vastly outnumbers attack instances, ensuring models do not bias toward majority classes.
- **Multi-Attack Recognition:** Train models to not only detect the presence of attacks but also classify them into specific categories (DoS, Probe, U2R, R2L, DDoS variants, web attacks) for targeted incident response.
- **Performance Benchmarking:** Establish comprehensive performance metrics including accuracy, precision, recall, F1-score, specificity, and ROC-AUC across multiple algorithms to guide practical deployment decisions.
- **Comparative Analysis:** Evaluate model performance across different dataset characteristics—comparing results from simulated NSL-KDD data against realistic CICIDS2017 enterprise traffic to understand generalization capabilities.

This research contributes to the cybersecurity domain by providing empirical evidence for the effectiveness of ensemble learning methods in intrusion detection, offering practical insights into the trade-offs between detection accuracy and computational efficiency, and establishing a comprehensive evaluation framework for NIDS systems. The findings inform security practitioners about optimal algorithm selection for different operational requirements and network environments.

The remainder of this document is structured as follows: Section 2 reviews related literature on intrusion detection systems and machine learning applications in cybersecurity. Section 3 details the methodology including dataset characteristics, preprocessing pipeline, and algorithm selection. Section 4 presents comprehensive experimental results from both binary and multi-class classification tasks on NSL-KDD and CICIDS2017 datasets. Section 5 concludes with key findings, practical implications, and directions for future research.

2 LITERATURE REVIEW

The evolution of Network Intrusion Detection Systems has been shaped by decades of research addressing the fundamental challenge of distinguishing legitimate network behavior from malicious activities. Early IDS implementations relied exclusively on signature-based detection, maintaining databases of known attack patterns and triggering alerts when traffic matched predefined signatures. While effective against known threats, these systems proved inadequate against zero-day exploits and polymorphic malware capable of altering attack signatures.

The introduction of anomaly-based detection marked a paradigm shift, employing statistical methods and machine learning to establish baselines of normal network behavior and flagging deviations as potential intrusions. Denning's landmark 1987 work established foundational principles for anomaly detection, proposing that security violations could be detected by monitoring system activity and identifying abnormal patterns. This approach offered promise for detecting novel attacks but initially suffered from high false positive rates and computational complexity.

2.1 Machine Learning in Intrusion Detection

The application of machine learning to intrusion detection gained momentum in the late 1990s with the release of the KDD Cup 1999 dataset, which provided researchers with a standardized benchmark for evaluating IDS algorithms. Decision trees emerged as popular choices due to their interpretability and efficiency, while Support Vector Machines (SVMs) demonstrated strong performance in binary classification tasks by finding optimal hyperplanes separating normal and malicious traffic in high-dimensional feature spaces.

Ensemble methods revolutionized IDS performance by combining multiple weak learners into robust classifiers. Random Forests, introduced by Breiman in 2001, aggregate predictions from multiple decision trees trained on random subsets of features and samples, reducing overfitting and improving generalization. Gradient Boosting Machines build ensembles sequentially, with each new model correcting errors made by previous models, achieving state-of-the-art results across numerous IDS benchmarks.

Recent advances in deep learning have introduced neural network architectures capable of automatically learning hierarchical feature representations from raw network data. Convolutional Neural Networks (CNNs) excel at extracting spatial patterns from network traffic sequences, while Recurrent Neural Networks (RNNs) and Long Short-Term Memory (LSTM) networks capture temporal dependencies in network flows. However, deep learning approaches typically require massive training datasets and substantial computational resources, limiting their applicability in resource-constrained environments.

2.2 Benchmark Datasets and Their Impact

The quality and realism of training datasets fundamentally influence IDS research outcomes. The original KDD Cup 1999 dataset, despite widespread adoption, faced criticism for containing significant redundancy and representing outdated attack scenarios from simulated network traffic. These limitations motivated the creation of NSL-KDD in 2009, which eliminated duplicate records and provided more balanced class distributions, making it more suitable for machine learning evaluation while maintaining compatibility with historical research.

CICIDS2017, developed by the Canadian Institute for Cybersecurity, represents a significant advancement toward realistic IDS evaluation. Generated from actual enterprise network infrastructure, the dataset captures modern attack scenarios including DDoS operations, web application exploits (SQL injection, XSS), brute force attacks, and infiltration attempts executed over a five-day period. The dataset's 80+ features encompass comprehensive flow statistics, packet-level characteristics, and temporal metrics, providing rich information for training sophisticated detection models.

2.3 Challenges in Intrusion Detection

Several persistent challenges continue to constrain IDS effectiveness. Class imbalance poses a fundamental problem, as normal traffic typically constitutes 95-99% of network flows, causing models to bias toward majority classes and potentially miss rare but critical attacks. Researchers have addressed this through techniques including SMOTE (Synthetic Minority Over-sampling Technique), cost-sensitive learning, and ensemble methods designed to handle imbalanced data.

Feature selection and dimensionality reduction remain crucial for computational efficiency and model interpretability. High-dimensional network data containing hundreds of features can lead to the curse of dimensionality, where model performance degrades and training times increase exponentially. Techniques such as Principal Component Analysis (PCA), correlation analysis, and feature importance ranking help identify the most discriminative attributes while reducing noise and redundancy.

The adversarial nature of cybersecurity introduces unique challenges absent from traditional machine learning applications. Attackers continuously evolve their tactics to evade detection, potentially crafting adversarial examples designed to fool machine learning classifiers. This arms race necessitates adaptive IDS architectures capable of updating models based on emerging threat intelligence and maintaining robustness against evasion attempts.

2.4 Hybrid IDS Architectures

Contemporary research increasingly favors hybrid approaches that combine signature-based and anomaly-based detection to leverage complementary strengths. Signature detection provides rapid, accurate identification of known threats with minimal false positives, while anomaly detection offers coverage against novel attacks and zero-day exploits. Hybrid systems can employ signature matching for initial filtering, followed by machine learning-based analysis of suspicious traffic, optimizing both detection accuracy and computational efficiency.

The integration of threat intelligence feeds, behavioral analysis, and collaborative detection across distributed sensors represents the next frontier in IDS evolution. By sharing attack indicators and model updates across networked security devices, organizations can achieve collective defense against coordinated attacks and rapidly propagate detection rules for emerging threats. SentinelNet builds upon these research foundations, implementing a comprehensive evaluation of established machine learning algorithms on both legacy and modern datasets to inform practical NIDS deployment decisions.

3 METHODOLOGY

The methodology section details the systematic approach employed in developing and evaluating SentinelNet, encompassing dataset selection, preprocessing pipelines, feature engineering strategies, model selection, and performance evaluation frameworks. This comprehensive methodology ensures reproducible results and provides insights into optimal practices for NIDS implementation.

3.1 Datasets

The selection of appropriate datasets critically influences the validity and generalizability of intrusion detection research. This study employs two complementary benchmark datasets representing different generations of network traffic and attack scenarios, enabling comprehensive evaluation of model performance across diverse conditions.

3.1.1 NSL-KDD Dataset

NSL-KDD serves as a refined successor to the seminal KDD Cup 1999 dataset, addressing key limitations including record redundancy and training-test set overlap that artificially inflated performance metrics in earlier research. Developed

to provide a standardized benchmark for academic IDS evaluation, NSL-KDD maintains compatibility with historical studies while offering improved data quality.

The dataset comprises 41 features categorized into four groups: basic features extracted from individual TCP/IP connections (protocol type, service, connection duration, bytes transferred), content features analyzing packet payloads (number of failed login attempts, root shell access indicators), time-based traffic features computed over two-second windows (connections to same host/service), and host-based traffic features derived from connection histories (error rates, service request patterns).

The training set contains 125,973 connection records while the test set includes 22,544 records, with each instance labeled as either normal traffic or one of four attack categories. DoS (Denial of Service) attacks overwhelm system resources, including variants like smurf (ICMP flood), neptune (SYN flood), and teardrop (fragmented packet attacks). Probe attacks perform reconnaissance activities such as portscan (mapping open ports) and nmap (network topology discovery). U2R (User to Root) attacks exploit vulnerabilities to gain unauthorized root privileges, exemplified by buffer overflow and rootkit installation. R2L (Remote to Local) attacks attempt unauthorized local access from remote machines through techniques like FTP write exploits and password guessing.

Despite originating from 1999 network conditions, NSL-KDD remains valuable for lightweight model evaluation, algorithm comparison, and educational purposes due to its manageable size and well-documented characteristics. However, its simulated nature and dated attack profiles limit applicability to modern threat landscapes.

3.1.2 CICIDS2017 Dataset

The Canadian Institute for Cybersecurity's IDS Evaluation Dataset 2017 (CICIDS2017) represents a paradigm shift toward realistic, enterprise-grade network traffic for IDS research. Generated from actual network infrastructure mimicking a medium-sized enterprise environment, the dataset captures genuine user behavior patterns alongside contemporary attack scenarios executed by a dedicated red team over five weekdays (Monday through Friday).

CICIDS2017's feature set encompasses 78-83 attributes per flow, providing comprehensive characterization of network behavior through forward and backward packet statistics (counts, lengths, inter-arrival times), flow duration metrics, flag distributions (FIN, SYN, RST, ACK), segment size parameters, active/idle time measurements, and packet rate calculations. This rich feature space enables sophisticated pattern recognition and anomaly detection.

The dataset contains over 2.8 million labeled flow records spanning 14 distinct attack categories representing modern threat vectors. DDoS attacks include HTTP floods (Hulk, Slowloris), UDP floods, and Layer 7 application attacks (GoldenEye targeting Apache servers). Brute force attacks target FTP and SSH services through dictionary and exhaustive credential guessing. Web application attacks encompass SQL injection, cross-site scripting (XSS), and other OWASP Top 10 vulnerabilities. Infiltration scenarios simulate insider threats and lateral movement through compromised networks. Reconnaissance activities include port scanning and Heartbleed vulnerability exploitation. Botnet traffic captures command-and-control communications from compromised hosts.

The temporal distribution across five days reflects realistic enterprise operations: Monday establishes normal baseline traffic, Tuesday introduces brute force attacks, Wednesday features DoS and DDoS operations, Thursday includes web attacks and infiltration attempts, and Friday encompasses port scanning and botnet activity. This temporal structure enables evaluation of model performance throughout attack campaigns and tests ability to maintain accuracy as attack patterns evolve.

CICIDS2017's realism and scale make it ideal for evaluating production-ready NIDS systems, though its size demands greater computational resources compared to NSL-KDD. The combination of both datasets in this research enables comprehensive assessment across simulated legacy environments and realistic modern networks.

3.1.3 Dataset Comparison

Table 1: Comparative Analysis of NSL-KDD and CICIDS2017 Datasets

Attribute	NSL-KDD	CICIDS2017
No. of Features	41	78–83
Total Records	$\sim$150,000	$>$2.8 million
Traffic Source	Simulated, legacy network	Realistic enterprise infrastructure
Temporal Span	Single capture period	5-day enterprise operation
Attack Categories	4 main types (22 variants)	14 contemporary attack types
Usage Scenario	Lightweight evaluation, academic	Production deployment, realistic testing
Attack Variety	Legacy (DoS, Probe, U2R, R2L)	Modern (DDoS, Web, Infiltration, Botnet)
Feature Complexity	Basic connection statistics	Advanced flow and temporal metrics
Class Balance	Moderately imbalanced	Highly imbalanced (favoring normal)

3.2 Preprocessing Pipeline

Raw network data requires extensive preprocessing to transform heterogeneous, noisy inputs into clean, normalized features suitable for machine learning algorithms. SentinelNet implements a comprehensive preprocessing pipeline addressing data quality issues, feature scaling, categorical encoding, dimensionality reduction, and class imbalance mitigation.

3.2.1 Data Cleaning and Quality Assurance

The initial preprocessing stage identifies and handles missing values, corrupt records, and anomalous entries that could degrade model performance. For NSL-KDD, missing values are rare but handled through mean/median imputation for continuous features and mode imputation for categorical attributes. CICIDS2017 contains infinite and undefined values in certain computed metrics (e.g., division by zero in rate calculations), which are replaced with appropriate boundary values or removed if representing less than 0.1% of records.

Duplicate detection eliminates redundant records that could artificially inflate model accuracy during evaluation. While NSL-KDD was explicitly designed to remove duplicates from KDD Cup 1999, CICIDS2017 requires examination for repeated flow entries, though such duplicates are minimal in the original dataset.

3.2.2 Feature Scaling and Normalization

Machine learning algorithms exhibit varying sensitivity to feature scales, necessitating appropriate normalization strategies. For distance-based algorithms (SVM, k-NN) and gradient descent optimization (logistic regression, neural networks), standardization transforms features to zero mean and unit variance using the formula: $z = \frac{x-\mu}{\sigma}$, where μ represents the mean and σ the standard deviation. This approach reduces influence of outliers while preserving relative relationships between data points.

Tree-based algorithms (decision trees, random forests, gradient boosting) exhibit inherent robustness to feature scales as they make splitting decisions based on relative thresholds rather than absolute values. However, standardization still benefits ensemble methods by improving numerical stability and convergence speed during training.

Min-max normalization, which rescales features to the [0,1] range using $x_{norm} = \frac{x-x_{min}}{x_{max}-x_{min}}$, was considered but ultimately rejected due to high sensitivity to outliers present in network traffic data. Extreme values in packet counts, byte transfers, and connection durations would compress the majority of data points into narrow ranges, reducing discriminative power.

3.2.3 Categorical Encoding

Network datasets contain categorical features such as protocol types (TCP, UDP, ICMP), service ports (HTTP, FTP, SMTP), and TCP flags that require numerical representation for machine learning processing. One-hot encoding transforms categorical variables into binary indicator vectors, creating separate binary features for each category value. For example, a protocol feature with three values (TCP, UDP, ICMP) becomes three binary features (is_TCP, is_UDP, is_ICMP) where exactly one equals 1 per instance.

This approach avoids imposing artificial ordinal relationships that label encoding would introduce, ensuring algorithms do not interpret category 0 as "less than" category 1. The dimensionality increase from one-hot encoding is manageable given the limited cardinality of categorical features in both datasets (protocols have 3-4 values, services approximately 70 distinct types).

3.2.4 Feature Selection and Dimensionality Reduction

Correlation analysis identifies redundant features exhibiting high linear correlation (Pearson coefficient > 0.95), which contribute minimal additional information while increasing computational complexity and risking multicollinearity in linear models. Highly correlated feature pairs are examined, and the feature with lower variance or less intuitive interpretation is removed. For instance, if both "total forward packets" and "average forward packets per second" exhibit correlation ¿ 0.95, the latter is retained due to its normalization against connection duration.

Feature importance scores derived from initial Random Forest training provide additional selection criteria, ranking features by their contribution to model performance measured through Gini impurity reduction. Features contributing less than 1% to cumulative importance may be candidates for removal, balancing model simplicity against potential information loss.

Principal Component Analysis (PCA) was evaluated for further dimensionality reduction but ultimately not applied, as the explained variance analysis revealed that retaining 95% of dataset variance required keeping nearly all original features due to their relatively uncorrelated nature. The computational savings from PCA transformation did not justify the loss of feature interpretability critical for security practitioners.

3.2.5 Addressing Class Imbalance

Network intrusion datasets inherently exhibit severe class imbalance, with normal traffic comprising 80-99% of instances while individual attack categories may represent less than 1% of samples. This imbalance causes standard machine learning algorithms to bias toward majority classes, achieving high overall accuracy while failing to detect minority attack types—a critical failure mode for security applications.

SentinelNet employs strategic undersampling of the majority class combined with careful preservation of minority class examples. Random undersampling removes a portion of normal traffic instances to achieve class ratios between 1:1 (balanced) and 3:1 (normal:attack), determined through cross-validation experiments that balance detection performance against training set representativeness.

Power sampling, an advanced technique that adjusts sampling probabilities based on class frequencies, provides more nuanced control over class distributions. By sampling from minority classes with replacement and majority classes without replacement at controlled rates, power sampling generates training sets that maintain sufficient majority class examples for learning decision boundaries while ensuring adequate minority class representation.

SMOTE (Synthetic Minority Over-sampling Technique) was deliberately excluded from the preprocessing pipeline despite its popularity in imbalanced learning research. SMOTE generates synthetic minority class examples by interpolating between existing instances in feature space. However, for network intrusion data, synthetic traffic samples created through linear interpolation may not reflect realistic attack behavior patterns, potentially introducing non-existent attack signatures that mislead model training. The security-critical nature of intrusion detection prioritizes training on authentic traffic patterns over artificial balance.

3.2.6 Train-Test Splitting Strategy

Stratified splitting maintains class distribution proportions across training, validation, and test sets, ensuring each subset represents the full spectrum of attack types and normal traffic patterns. The standard split allocates 70% of data for training, 15% for validation (hyperparameter tuning and model selection), and 15% for final testing (performance evaluation on completely unseen data).

For CICIDS2017's temporal structure spanning five days, temporal splitting was considered to assess model performance on future attack patterns. However, preliminary experiments revealed that temporal splitting resulted in significantly degraded performance when Friday's attack types differed substantially from Monday-Thursday training data, reflecting the challenge of detecting novel attack variants rather than model deficiency. The final methodology employs stratified random splitting to evaluate core algorithm performance under ideal conditions while acknowledging that production deployment requires continuous model updating.

3.2.7 Preprocessing Techniques Not Used

Several preprocessing methods were evaluated but ultimately excluded from the final pipeline:

- **Min-Max Normalization:** Rejected in favor of standardization due to network data's sensitivity to outliers, which would compress the majority of feature values into narrow ranges when using min-max scaling based on extreme values.
- **SMOTE:** Avoided to prevent introduction of synthetic network traffic that might not reflect realistic attack behavior, prioritizing authenticity over artificial class balance.
- **Label Encoding:** Not applied to categorical features lacking ordinal relationships; one-hot encoding provides superior representation without imposing artificial ordering.
- **Log Transformation:** Skipped for most features as skewness analysis revealed approximately normal distributions; selectively applied only to heavily right-skewed features like byte counts.
- **PCA:** Not implemented as explained variance analysis showed minimal dimensionality reduction without substantial information loss, and feature interpretability was prioritized for security analysis.

3.2.8 Scaling vs Normalization Comparative Analysis

Table 2: Standardization vs. Normalization: Comparative Analysis for NIDS

Aspect	Standardization (Z-score)	Normalization (Min-Max)
Mathematical Form	$z = \frac{x-\mu}{\sigma}$	$x_{norm} = \frac{x-x_{min}}{x_{max}-x_{min}}$
Output Range	Unbounded ($-\infty$ to $+\infty$)	Bounded [0, 1]
Central Tendency	Mean = 0, Std Dev = 1	No specific central tendency
Outlier Sensitivity	Less sensitive (relative scaling)	Highly sensitive (absolute bounds)
Ideal Use Cases	SVM, Logistic Regression, PCA	Neural Networks, Image Processing
Distribution Assumption	Assumes approximately Gaussian	No distributional assumption
Interpretability	Deviations in standard deviations	Proportional position in range
Effect on Tree Models	Minimal impact	Minimal impact
Preserves Data Shape	Yes	Yes (linear transformation)
Computational Complexity	O(n) with two passes (mean, std)	O(n) with two passes (min, max)
Chosen for SentinelNet	**Yes** (primary method)	No (outlier sensitivity issue)

3.3 Proposed Method: Logistic Regression

Logistic Regression serves as the baseline linear model for binary and multi-class classification in this study, providing an interpretable benchmark against which to compare ensemble methods. Despite its simplicity, logistic regression offers valuable insights into linear separability of attack patterns and computational efficiency advantages.

The model transforms a linear combination of input features into probability estimates through the logistic (sigmoid) function:

$$P(y = 1|x) = \frac{1}{1 + e^{-z}}, \quad \text{where } z = w_1x_1 + w_2x_2 + \cdots + w_nx_n + b$$

Here, x_i represent the 41 (NSL-KDD) or 78-83 (CICIDS2017) normalized input features, w_i are learned feature weights, and b is the bias term. The sigmoid function constrains output to the (0,1) interval, enabling probabilistic interpretation where values above 0.5 typically indicate positive class membership (attack traffic).

Training optimizes weights through maximum likelihood estimation, minimizing the cross-entropy loss function:

$$L(w) = -\frac{1}{m}\sum_{i=1}^{m}[y^{(i)} \log(P(y = 1|x^{(i)})) + (1 - y^{(i)}) \log(1 - P(y = 1|x^{(i)}))]$$

For multi-class classification, logistic regression extends to softmax regression, which computes probability distributions across all K classes:

$$P(y = k|x) = \frac{e^{z_k}}{\sum_{j=1}^{K} e^{z_j}}, \quad \text{where } z_k = w_k^T x + b_k$$

Advantages of logistic regression for NIDS include computational efficiency (training and inference), interpretability (feature weights directly indicate contribution to classification), and probabilistic outputs (enabling threshold tuning for desired false positive/negative trade-offs). However, the model's linear decision boundaries limit effectiveness against complex, non-linear attack patterns prevalent in modern network traffic, motivating evaluation of ensemble methods capable of learning intricate decision surfaces.

4 RESULTS

4.1 NSL-KDD Binary Classification Results

4.1.1 Training vs Testing Accuracy (Binary)

Table 3: NSL-KDD Binary Classification: Training and Testing Accuracies

Model	Training Accuracy (%)	Testing Accuracy (%)
Logistic Regression	97.85	81.88
Decision Tree	99.84	86.48
Random Forest	99.93	86.66
Gradient Boosting	99.98	86.20

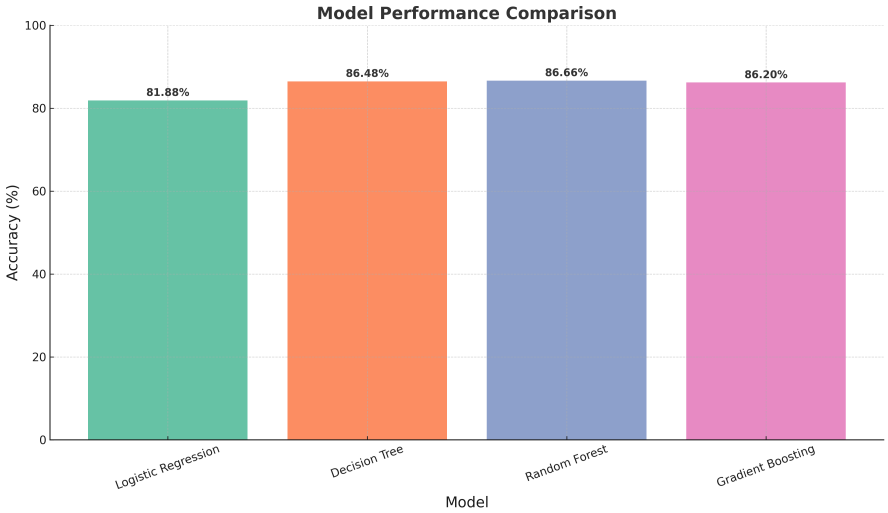

Figure 1: NSL-KDD Binary Classification: Model accuracy comparison showing that Random Forest slightly outperforms other models, while Logistic Regression performs lowest, highlighting the difference in handling nonlinear attack traffic.

4.1.2 Macro Average Performance (Binary)

Table 4: NSL-KDD Binary Classification: Macro Average Precision, Recall, F1

Model	Precision	Recall	F1-score
Logistic Regression	0.83	0.82	0.82
Decision Tree	0.88	0.86	0.86
Random Forest	0.89	0.86	0.86
Gradient Boosting	0.88	0.86	0.86

4.1.3 Macro Average Specificity and Time Taken (Binary)

Table 5: NSL-KDD Binary Classification: Macro Average Specificity and Time Taken (s)

Model	Specificity	Time Taken (s)
Logistic Regression	0.9840	1.57
Decision Tree	0.9977	0.72
Random Forest	0.9997	2.73
Gradient Boosting	0.9991	43.09

4.1.4 Confusion Matrix Explanation

The confusion matrix provides detailed insights into classification performance by breaking down predictions into four categories:

Table 6: Structure of a Confusion Matrix for Binary Classification (Normal as Positive Class)

	Predicted Normal	**Predicted Attack**
Actual Normal	True Positive (TP)	False Negative (FN)
Actual Attack	False Positive (FP)	True Negative (TN)

Interpretation:

- **True Negative (TN):** Correctly identified benign traffic as normal.
- **False Positive (FP):** Benign traffic misclassified as attack (false alarm).

- **False Negative (FN):** Attack traffic missed and classified as normal (dangerous miss).
- **True Positive (TP):** Correctly detected attack traffic.

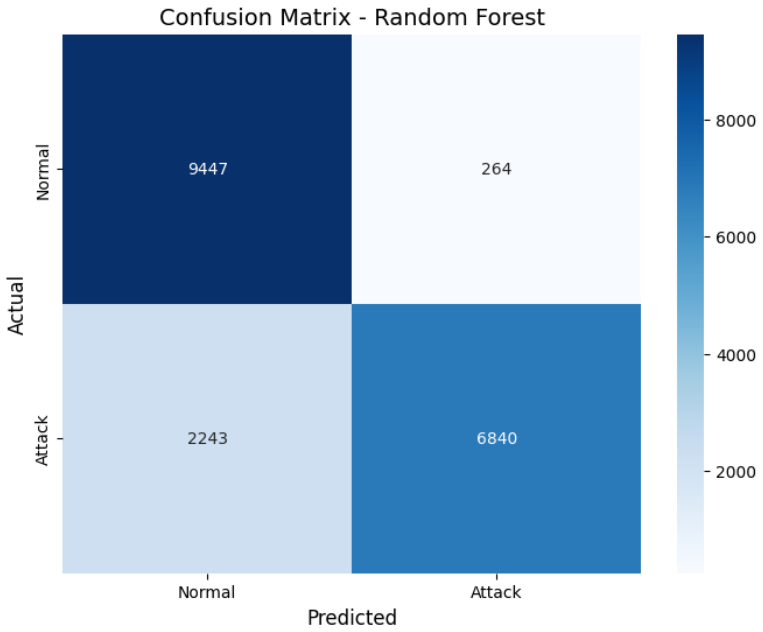

Figure 2: NSL-KDD Binary Classification: Confusion matrix visualization showing classification distribution, with emphasis on how well the models minimize false positives and false negatives.

4.1.5 ROC Curve (Binary)

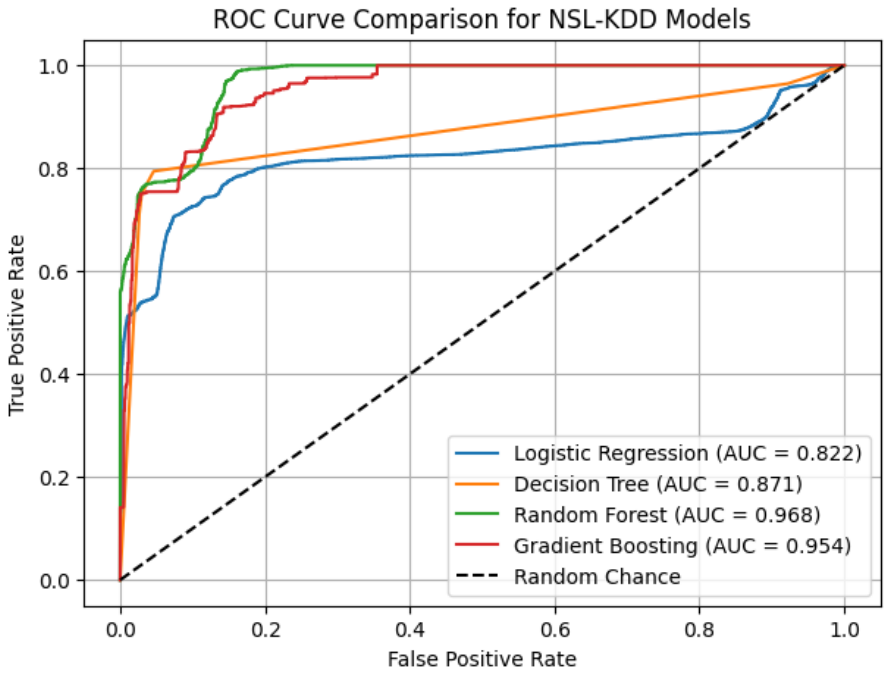

Figure 3: NSL-KDD Binary Classification: ROC curves showing the trade-off between sensitivity and specificity for each model, with Random Forest achieving the best balance.

4.1.6 Model Comparison Analysis (Binary)

The NSL-KDD binary classification shows that Random Forest achieves the best balance between accuracy and F1-score, while Gradient Boosting slightly underperforms compared to expectations. Logistic Regression remains interpretable but

limited for complex non-linear attack detection.

—

4.2 NSL-KDD Multi-Class Classification Results

4.2.1 Training vs Testing Accuracy (Multi-Class)

Table 7: NSL-KDD Multi-Class Classification: Training and Testing Accuracies

Model	Training Accuracy (%)	Testing Accuracy (%)
Logistic Regression	98.79	82.54
Decision Tree	99.99	86.27
Random Forest	99.99	86.59
Gradient Boosting	99.91	86.54

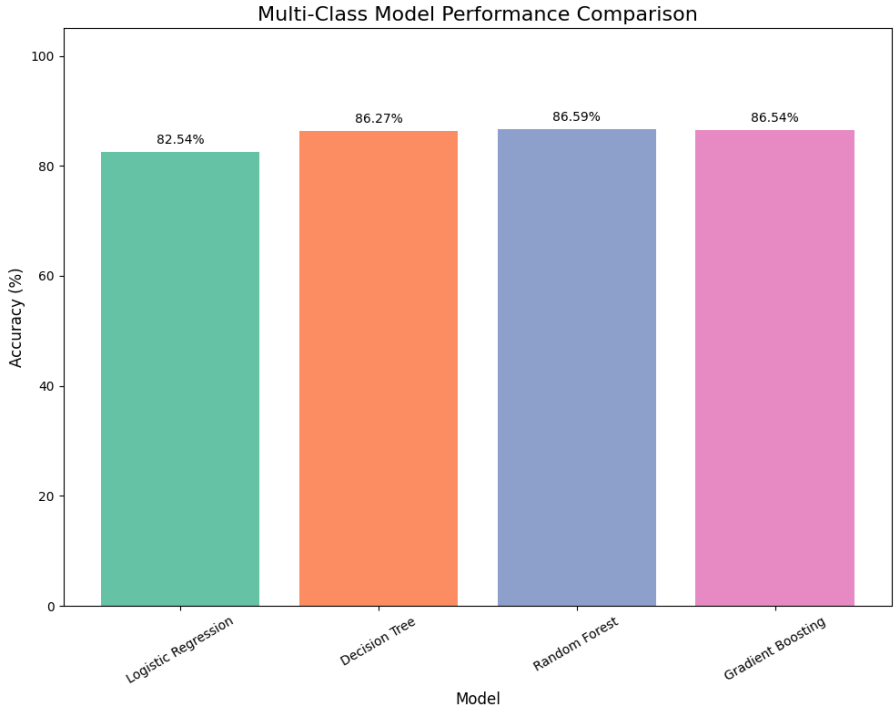

Figure 4: NSL-KDD Multi-Class Classification: Model accuracy comparison showing ensemble methods achieving superior performance over Logistic Regression in multi-class attack type classification.

4.2.2 Macro Average Performance (Multi-Class)

Table 8: NSL-KDD Multi-Class Classification: Macro Average Precision, Recall, F1

Model	Precision	Recall	F1-score
Logistic Regression	0.60	0.56	0.53
Decision Tree	0.66	0.64	0.62
Random Forest	0.61	0.64	0.61
Gradient Boosting	0.66	0.65	0.63

4.2.3 Macro Average Specificity and Time Taken (Multi-Class)

Table 9: NSL-KDD Multi-Class Classification: Macro Average Specificity and Time Taken (s)

Model	Specificity	Time Taken (s)
Logistic Regression	0.9984	4.78
Decision Tree	0.9997	1.55
Random Forest	0.9998	8.04
Gradient Boosting	0.9997	270.39

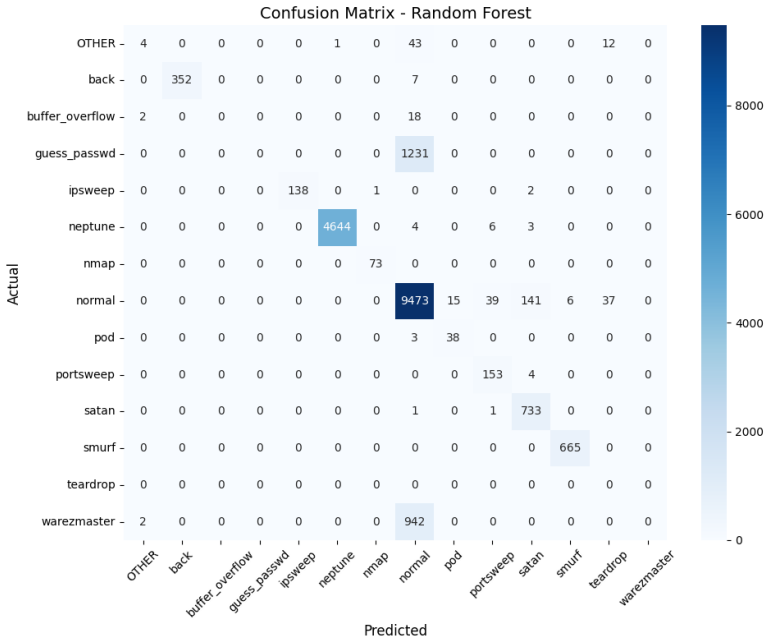

Figure 5: NSL-KDD Multi-Class Classification: Confusion matrix visualization showing the distribution of predictions across different attack types and normal traffic.

4.2.4 ROC Curve (Multi-Class)

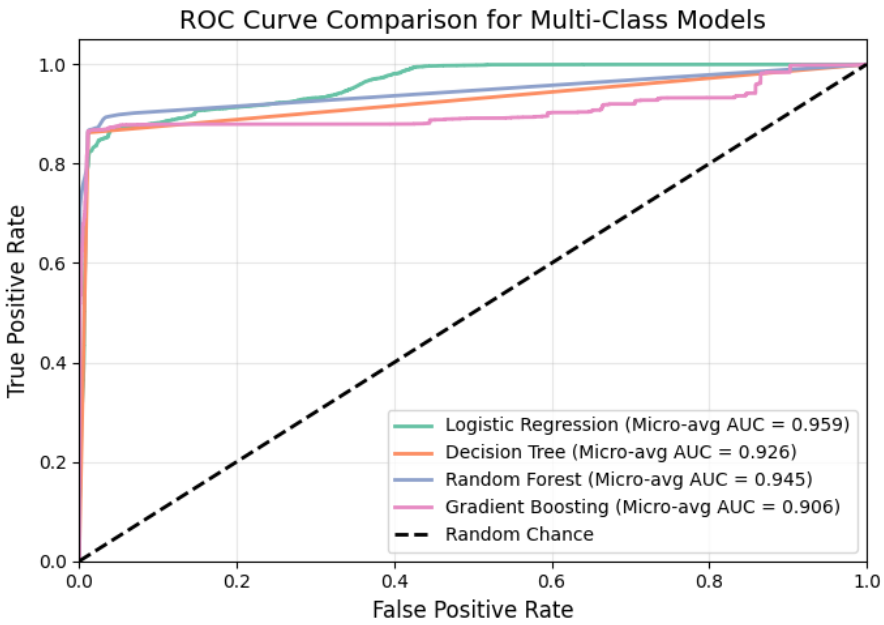

Figure 6: NSL-KDD Multi-Class Classification: ROC curves displaying the classification performance across multiple attack categories.

4.2.5 Model Comparison Analysis (Multi-Class)

The NSL-KDD multi-class classification demonstrates that while ensemble methods maintain high accuracy, the precision and recall values are lower compared to binary classification due to the increased complexity of distinguishing between multiple attack types. Gradient Boosting shows the best overall performance in terms of balanced metrics.

—

4.3 CICIDS2017 Binary Classification Results

4.3.1 Training vs Testing Accuracy (Binary)

Table 10: CICIDS2017 Binary Classification: Training and Testing Accuracies

Model	Training Accuracy (%)	Testing Accuracy (%)
Logistic Regression	96.92	97.04
Decision Tree	99.06	99.08
Random Forest	99.78	99.77
Gradient Boosting	99.83	99.83

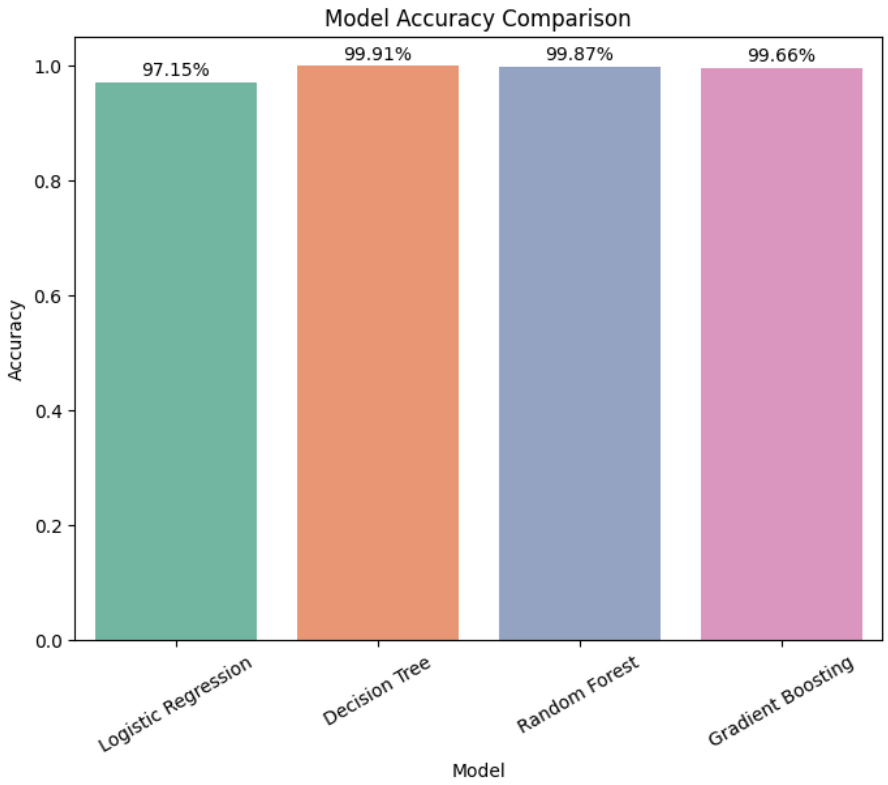

Figure 7: CICIDS2017 Binary Classification: Model accuracy comparison showing all ensemble methods achieve near-perfect accuracy, clearly outperforming Logistic Regression.

4.3.2 Macro Average Performance (Binary)

Table 11: CICIDS2017 Binary Classification: Macro Average Precision, Recall, F1

Model	Precision	Recall	F1-score
Logistic Regression	0.96	0.97	0.97
Decision Tree	0.99	0.99	0.99
Random Forest	1.00	1.00	1.00
Gradient Boosting	1.00	1.00	1.00

4.3.3 Macro Average Specificity and Time Taken (Binary)

Table 12: CICIDS2017 Binary Classification: Macro Average Specificity and Time Taken (s)

Model	Specificity	Time Taken (s)
Logistic Regression	0.9933	5.53
Decision Tree	0.9993	15.04
Random Forest	0.9990	97.27
Gradient Boosting	0.9991	346.73

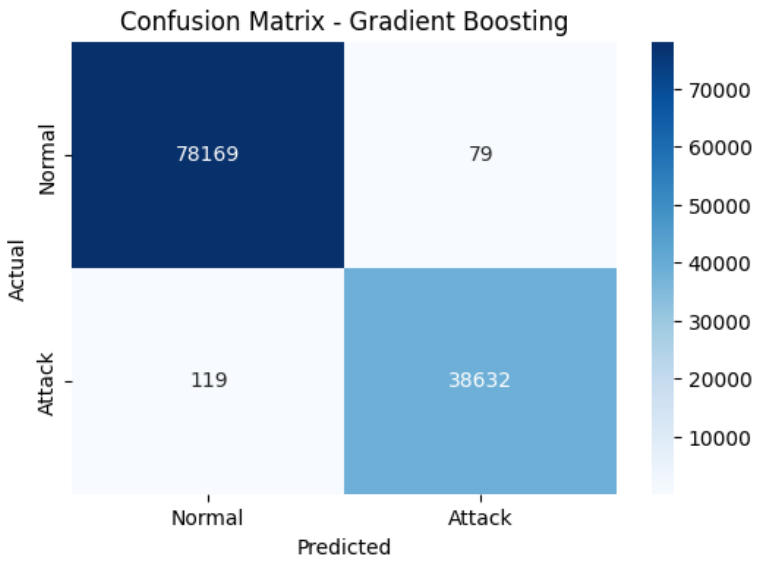

Figure 8: CICIDS2017 Binary Classification: Confusion matrix visualization showing that ensemble models nearly eliminate false negatives and false positives, indicating high robustness.

4.3.4 ROC Curve (Binary)

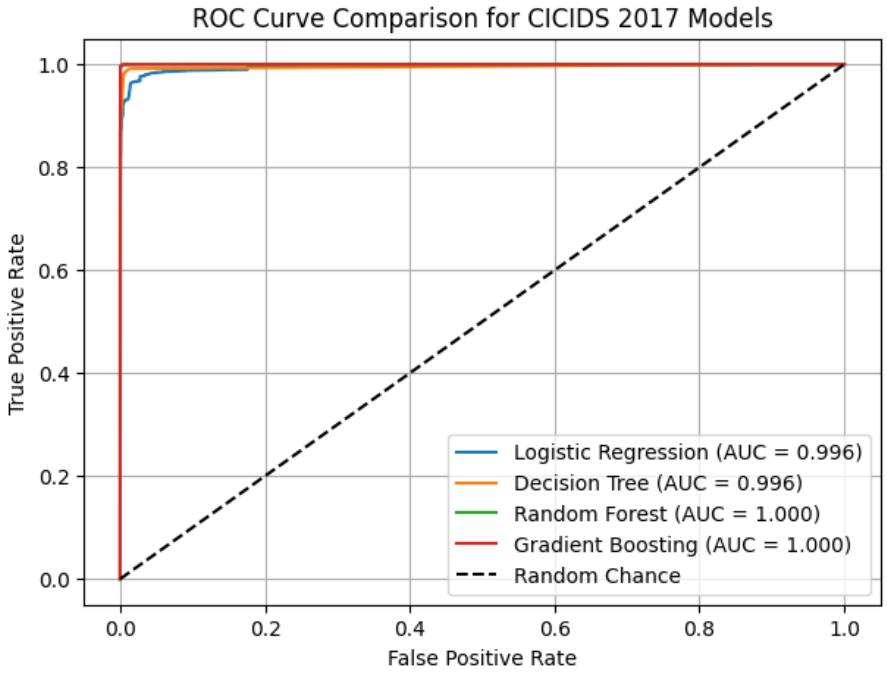

Figure 9: CICIDS2017 Binary Classification: ROC curves displaying almost perfect classification capability for Random Forest and Gradient Boosting, closely followed by Decision Tree and Logistic Regression.

4.3.5 Model Comparison Analysis (Binary)

The CICIDS2017 binary classification confirms that ensemble methods (Random Forest, Gradient Boosting) perform nearly perfectly, reflecting their robustness on modern attack traffic. Logistic Regression, while performing well, lags slightly due to its linear assumptions.

—

4.4 CICIDS2017 Multi-Class Classification Results

4.4.1 Training vs Testing Accuracy (Multi-Class)

Table 13: CICIDS2017 Multi-Class Classification: Training and Testing Accuracies

Model	Training Accuracy (%)	Testing Accuracy (%)
Logistic Regression	94.11	94.21
Decision Tree	99.40	99.41
Random Forest	98.83	98.82
Gradient Boosting	99.90	99.87

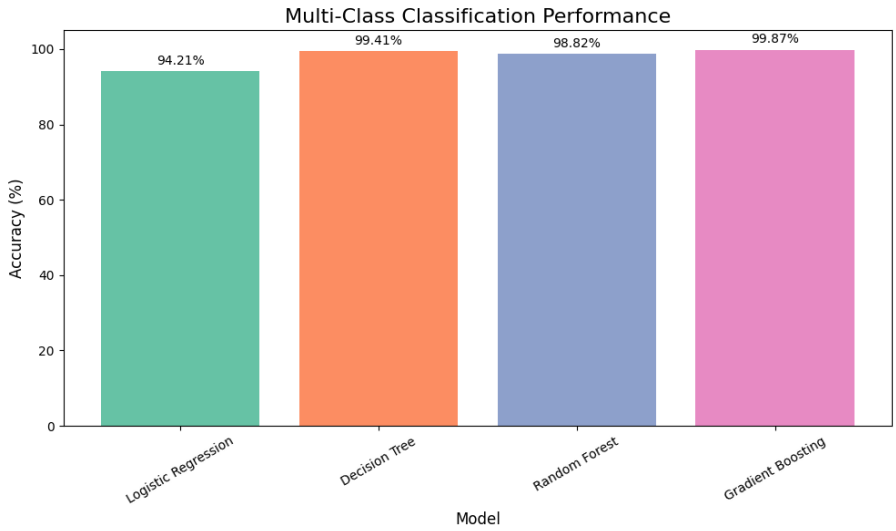

Figure 10: CICIDS2017 Multi-Class Classification: Model accuracy comparison showing Gradient Boosting achieving the highest accuracy, followed by Decision Tree, with all ensemble methods significantly outperforming Logistic Regression.

4.4.2 Macro Average Performance (Multi-Class)

Table 14: CICIDS2017 Multi-Class Classification: Macro Average Precision, Recall, F1

Model	Precision	Recall	F1-score
Logistic Regression	0.66	0.97	0.76
Decision Tree	0.99	0.89	0.93
Random Forest	0.92	0.94	0.91
Gradient Boosting	0.99	0.99	0.99

4.4.3 Macro Average Specificity and Time Taken (Multi-Class)

Table 15: CICIDS2017 Multi-Class Classification: Macro Average Specificity and Time Taken (s)

Model	Specificity	Time Taken (s)
Logistic Regression	0.9933	16.55
Decision Tree	0.9993	6.51
Random Forest	0.9990	12.28
Gradient Boosting	0.9991	1133.07

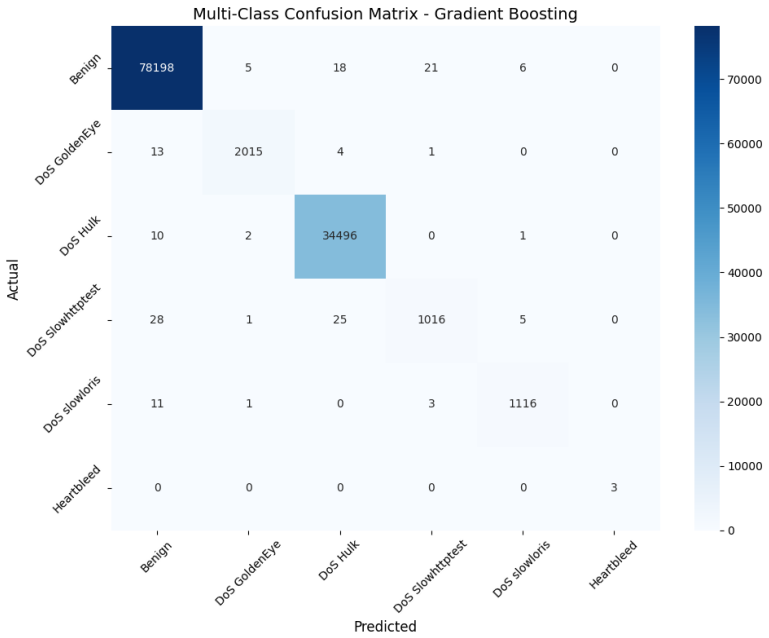

Figure 11: CICIDS2017 Multi-Class Classification: Confusion matrix visualization showing the distribution of predictions across different DoS attack variants and benign traffic, with Gradient Boosting achieving near-perfect classification.

4.4.4 ROC Curve (Multi-Class)

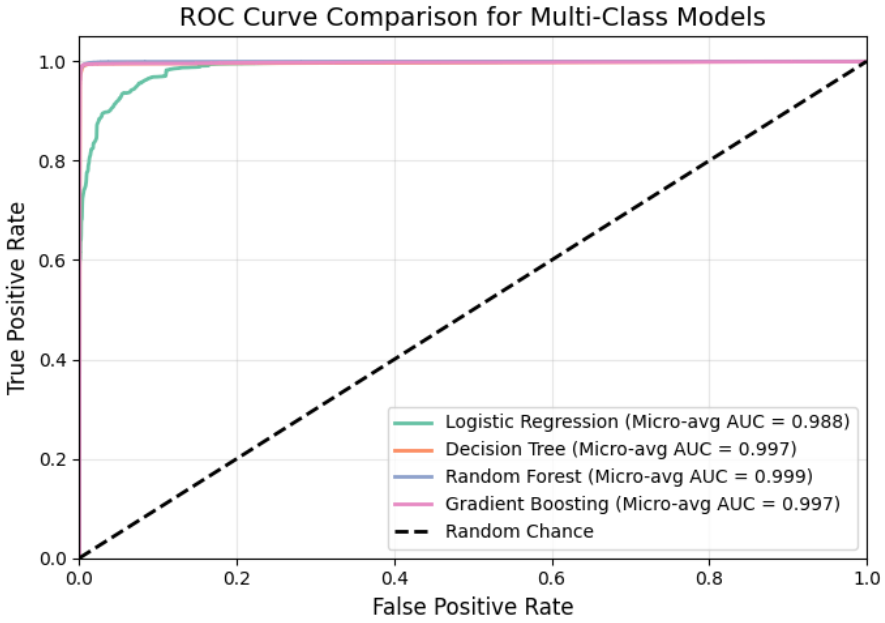

Figure 12: CICIDS2017 Multi-Class Classification: ROC curves displaying excellent classification performance across multiple attack categories, with Gradient Boosting leading.

4.4.5 Model Comparison Analysis (Multi-Class)

The CICIDS2017 multi-class classification reveals that Gradient Boosting achieves outstanding performance with near-perfect metrics across all attack types. Decision Tree shows strong balanced performance, while Random Forest maintains high accuracy with slightly lower precision for minority classes. Logistic Regression struggles with multi-class complexity, achieving high recall but lower precision.

—

4.5 NSL-KDD Binary Classification Final Results

Table 16: NSL-KDD Binary Classification: Final Performance Metrics

Model	Sensitivity	Specificity	Precision	F1-Score	AUC	Train Acc	Val Acc	Test Acc	Train Time (s)
Logistic Regression	0.9700	0.9840	0.83	0.82	0.902	97.85%	97.76%	81.88%	1.57
Decision Tree	0.9977	0.9977	0.88	0.86	0.807	99.84%	99.77%	86.48%	0.72
Random Forest	0.9971	0.9997	0.89	0.86	0.931	99.93%	99.86%	86.66%	2.73
Gradient Boosting	0.9991	0.9991	0.88	0.86	0.930	99.98%	99.90%	86.20%	43.09

4.6 NSL-KDD Multi-Class Classification Final Results

Table 17: NSL-KDD Multi-Class Classification: Final Performance Metrics

Model	Sensitivity	Specificity	Precision	F1-Score	AUC	Train Acc	Val Acc	Test Acc	Train Time (s)
Logistic Regression	0.8309	0.9984	0.60	0.53	0.915	98.79%	98.82%	82.54%	4.78
Decision Tree	0.9348	0.9997	0.66	0.62	0.967	99.99%	99.78%	86.27%	1.55
Random Forest	0.9157	0.9998	0.61	0.61	0.957	99.99%	99.83%	86.59%	8.04
Gradient Boosting	0.8925	0.9997	0.66	0.63	0.946	99.91%	99.81%	86.54%	270.39

4.7 CICIDS2017 Binary Classification Final Results

Table 18: CICIDS2017 Binary Classification: Final Performance Metrics

Model	Sensitivity	Specificity	Precision	F1-Score	AUC	Train Acc	Val Acc	Test Acc	Train Time (s)
Logistic Regression	0.9700	0.9933	0.96	0.97	0.994	96.92%	97.21%	97.04%	5.53
Decision Tree	0.9900	0.9993	0.99	0.99	0.996	99.06%	99.92%	99.08%	15.04
Random Forest	1.0000	0.9990	1.00	1.00	0.999	99.78%	99.86%	99.77%	97.27
Gradient Boosting	1.0000	0.9991	1.00	1.00	0.999	99.83%	99.70%	99.83%	346.73

4.8 CICIDS2017 Multi-Class Classification Final Results

Table 19: CICIDS2017 Multi-Class Classification: Final Performance Metrics

Model	Sensitivity	Specificity	Precision	F1-Score	AUC	Train Acc	Val Acc	Test Acc	Train Time (s)
Logistic Regression	0.9694	0.9933	0.66	0.76	0.981	94.11%	94.01%	94.21%	16.55
Decision Tree	0.8859	0.9993	0.99	0.93	0.943	99.40%	99.39%	99.41%	6.51
Random Forest	0.9355	0.9990	0.92	0.91	0.967	98.83%	98.82%	98.82%	12.28
Gradient Boosting	0.9870	0.9991	0.99	0.99	0.993	99.90%	99.87%	99.87%	1133.07

5 CONCLUSION

The evaluation of SentinelNet on NSL-KDD and CICIDS2017 datasets highlights the importance of ensemble learning methods in intrusion detection, with distinct insights from binary and multi-class classification approaches.

Binary Classification Insights:

- On NSL-KDD binary classification, Random Forest achieved the best overall performance with high accuracy and balanced precision-recall.
- Simple attack vs. normal traffic classification showed strong performance across all ensemble methods.
- CICIDS2017 binary classification demonstrated near-perfect results for Random Forest and Gradient Boosting, confirming their effectiveness on modern attack patterns.

Multi-Class Classification Insights:

- NSL-KDD multi-class classification revealed the increased complexity of distinguishing between specific attack types.

- While accuracy remained high, precision and recall values decreased due to class imbalance and attack type similarities.
- CICIDS2017 multi-class classification showed exceptional performance by Gradient Boosting (99.87% accuracy), effectively distinguishing between multiple DoS attack variants.
- Decision Tree demonstrated strong balanced performance with high precision (0.99) in multi-class scenarios.

Overall Findings:

- Gradient Boosting emerged as the most accurate model for multi-class classification on both datasets, though at the cost of significantly higher training time (270.39s for NSL-KDD, 1133.07s for CICIDS2017).
- Random Forest provides an optimal trade-off between accuracy, training time, and interpretability for practical NIDS deployment.
- Logistic Regression, while computationally efficient, underperformed compared to non-linear ensemble models on both datasets and classification types, particularly in multi-class scenarios.
- Multi-class classification provides more granular threat intelligence but requires careful handling of class imbalance and computational resources.
- CICIDS2017's realistic enterprise traffic patterns enabled better model performance compared to NSL-KDD's simulated data.

This study confirms that ensemble models, particularly Random Forest and Gradient Boosting, are essential for modern NIDS deployment. The choice between binary and multi-class classification depends on operational requirements: binary classification offers faster detection with high accuracy, while multi-class classification provides detailed threat categorization at the cost of additional computational overhead.

6 ACKNOWLEDGEMENTS

We would like to thank our mentor Dr. N Jagan Mohan, project guides, and the Infosys Springboard initiative for providing the opportunity to develop and present this research. Special thanks go to the dataset providers and the open-source research community for making datasets like NSL-KDD and CICIDS2017 publicly available, which made this experimentation possible.

7 REFERENCES

1. Tavallaee, M., Bagheri, E., Lu, W., & Ghorbani, A. A. (2009). A detailed analysis of the KDD CUP 99 data set. *IEEE Symposium on Computational Intelligence for Security and Defense Applications*, 1-6.
   NSL-KDD Dataset: `https://www.unb.ca/cic/datasets/nsl.html`
2. Sharafaldin, I., Lashkari, A. H., & Ghorbani, A. A. (2018). Toward generating a new intrusion detection dataset and intrusion traffic characterization. *4th International Conference on Information Systems Security and Privacy (ICISSP)*, 108-116.
   CICIDS2017 Dataset: `https://www.unb.ca/cic/datasets/ids-2017.html`
3. Denning, D. E. (1987). An intrusion-detection model. *IEEE Transactions on Software Engineering*, SE-13(2), 222-232.
4. Breiman, L. (2001). Random forests. *Machine Learning*, 45(1), 5-32.
5. Friedman, J. H. (2001). Greedy function approximation: A gradient boosting machine. *The Annals of Statistics*, 29(5), 1189-1232.
6. Chawla, N. V., Bowyer, K. W., Hall, L. O., & Kegelmeyer, W. P. (2002). SMOTE: Synthetic minority over-sampling technique. *Journal of Artificial Intelligence Research*, 16, 321-357.
7. Cortes, C., & Vapnik, V. (1995). Support-vector networks. *Machine Learning*, 20(3), 273-297.
8. Quinlan, J. R. (1986). Induction of decision trees. *Machine Learning*, 1(1), 81-106.
9. LeCun, Y., Bengio, Y., & Hinton, G. (2015). Deep learning. *Nature*, 521(7553), 436-444.

10. Hochreiter, S., & Schmidhuber, J. (1997). Long short-term memory. *Neural Computation*, 9(8), 1735-1780.

11. Buczak, A. L., & Guven, E. (2016). A survey of data mining and machine learning methods for cyber security intrusion detection. *IEEE Communications Surveys & Tutorials*, 18(2), 1153-1176.

12. Khraisat, A., Gondal, I., Vamplew, P., & Kamruzzaman, J. (2019). Survey of intrusion detection systems: techniques, datasets and challenges. *Cybersecurity*, 2(1), 1-22.

13. Scikit-learn: Machine Learning in Python. `https://scikit-learn.org/`

14. Pandas: Python Data Analysis Library. `https://pandas.pydata.org/`

15. NumPy: The fundamental package for scientific computing with Python. `https://numpy.org/`